P0156R0
Mike Spertus, Symantec
mike_spertus@symantec.com
revision of N4498
2015-10-21

Variadic `lock_guard` (Rev. 3)

The basic idea of this proposal is that `std::lock_guard` would benefit from being variadic to support multiple locks analogously to how `std::lock` does.

`lock_guard` is a very useful and widely used way to manage the lifetimes of lock ownership.

```
std::mutex mtx;
void f(){
  std::lock_guard<mutex> lck(mtx); // Mutex will be unlocked however scope is exited
  /* ... */
}
```

Unfortunately, if more than one lock needs to be required, life gets unnecessarily complicated

```
void swap(MyType const &l, MyType const &r)
{
    std::lock(l.mtx, r.mtx);
    std::lock_guard<std::mutex> llck(l.mtx, std::adopt_lock);
    std::lock_guard<std::mutex> rlck(r.mtx, std::adopt_lock);
    /* ... */
}
```

This is a lot more advanced and error-prone than the single lock case. For example, you often see the deadlock-inviting

```
void swap(MyType const &l, MyType const &r)
{
    std::lock_guard<std::mutex> llck(l.mtx);
    std::lock_guard<std::mutex> rlck(r.mtx);
    /* ... */
}
```

or the exception-unsafe

```
void swap(MyType const &l, MyType const &r)
{
    std::lock(l.mtx, r.mtx);
    /* ... */
    l.mtx.unlock();
    r.mtx.unlock();
}
```

These are exactly the kinds of complexities that are easily avoided by `std::lock_guard` in the single-lock case. Wouldn't it be great if `std::lock_guard` was variadic so it also worked with multiple locks?

```
void swap(MyType const &l, MyType const &r)
{
    std::lock_guard<std::mutex, std::mutex> lck(l.mtx, r.mtx);
    /* ... */
}
```

And that is exactly what we are proposing!

This can work in the presence of shared locking as well (just like `std::lock` does) as shown in the following example shared by Howard Hinnant

```
#include <mutex>
#include <shared_mutex>

class X
{
    using Mutex = std::shared_timed_mutex;
    using ReadLock = std::shared_lock<Mutex>;

    mutable Mutex mut_;
    // more data
public:
    // ...
```

```
    X& operator=(const X& x)
    {
        if (this !=&x)
        {
            ReadLock  rl(x.mut_, std::defer_lock);
            std::lock_guard<Mutex, ReadLock> lck(mut_, rl);
            // assign data ...
        }
        return *this;
    }
    // ...
};
```

which improves clarity while eliminating three lines of code

We do not propose creating a “make function” because `std::lock_guard` is not moveable (which we suspect is the reason `std::lock_guard` didn't have a make function in the first place), because we believe the two-lock case is most important in practice, because we want to keep this proposal simple, and because something along the lines of N4498 may offer a more general solution that obviates the entire issue. For now, we think that the already useful `std::lock_guard` class is strictly better when variadic.

Wording

Modify §30.4 [thread.mutex] as follows

```
template <class... MutexTypes> class lock_guard;
```

Modify §30.4.2.1 [thread.lock.guard] as follows

```
namespace std {
~~template <class Mutex>~~
  template <class... MutexTypes>
  class lock_guard {
  public:
    typedef Mutex mutex_type;  // If MutexTypes... consists of the single type Mutex

    ~~explicit lock_guard(mutex_type& m);~~
    ~~lock_guard(mutex_type& m, adopt_lock_t);~~
    explicit lock_guard(MutexTypes&... m);
    lock_guard(MutexTypes&... m, adopt_lock_t);
    ~lock_guard();

    lock_guard(lock_guard const&) = delete;
    lock_guard& operator=(lock_guard const&) = delete;
  private:
    ~~mutex_type&~~tuple<MutexTypes&...> pm; // exposition only
  };
}
```

An object of type `lock_guard` controls the ownership of ~~a~~ lockable objects within a scope. A `lock_guard` object maintains ownership of ~~a~~ lockable objects throughout the `lock_guard` object's lifetime (3.8). The behavior of a program is undefined if the lockable objects referenced by `pm` do~~es~~ not exist for the entire lifetime of the `lock_guard` object. When `sizeof...(MutexTypes)` is 1, the supplied `Mutex` type shall meet the `BasicLockable` requirements. Otherwise, each of the mutex types shall meet the `Lockable` requirements. (30.2.5.2).

```
explicit lock_guard(~~mutex_type&~~MutexTypes&... m);
```

Requires: If ~~`mutex_type`~~a `MutexTypes` type is not a recursive mutex, the calling thread does not own the corresponding mutex element of `m`.
*Effects:*Initializes `pm` with `tie(m...)`. Then if `sizeof...(MutexTypes)` is 0, no effects. Otherwise if `sizeof...(MutexTypes)` is 1, then `m.lock()`. Otherwise, then `lock(m...)`.
~~*Postcondition:* `&pm == &m`~~

```
lock_guard(~~mutex_type&~~MutexTypes&... m, adopt_lock_t);
```

Requires: The calling thread owns all the mutexes in `m`.
*Effects:*Initializes `pm` with `tie(m...)`.
~~*Postcondition:* `&pm == &m`~~
Throws: Nothing.

```
~lock_guard();
```

Effects: ~~`pm.unlock()`~~ For all `i` in `[0, sizeof...(MutexTypes))`, `get<i>(pm).unlock()`

Appendix: Example implementation

We interrupt this proposal for a message from our sponsor (N4064)

This proposal is a great advertisement for how the seemingly technical N4064 on improving `pair` and `tuple` can make trivial what would otherwise take some tricky metaprogramming. With N4064, the implementation could be as simple as

```
template<typename ...Ts>
struct lock_guard {
 explicit lock_guard(Ts&... ts) : mutexes(ts...) {}
 lock_guard(Ts&... ts, std::adopt_lock_t) : mutexes(ts..., std::adopt_lock_t()) {}
 tuple<lock_guard<Ts>...> mutexes;
};

template<typename T>
struct lock_guard<T> { /* As in C++14 */ };
```

We now return to our regularly scheduled proposal

Without the fixes in N4064, the best way to implement seems to be to use the `for_each_in_tuple` infrastructure posted by Andy Prowl on Stack Overflow to lock and unlock all the mutexes in the `tuple`.

```
#include<mutex>
#include<tuple>
using std::tuple;
// Taken from http://stackoverflow.com/questions/16387354/template-tuple-calling-a-function-on-each-element
namespace detail
{
  template<int... Is>
  struct seq { };

  template<int N, int... Is>
  struct gen_seq : gen_seq<N - 1, N - 1, Is...> { };

  template<int... Is>
  struct gen_seq<0, Is...> : seq<Is...>{};

  template<typename T, typename F, int... Is>
  void for_each(T&& t, F f, seq<Is...>) {
    auto l = { (f(std::get<Is>(t)), 0)... };
  }
}

template<typename... Ts, typename F>
void for_each_in_tuple(std::tuple<Ts...> const& t, F f) {
  detail::for_each(t, f, detail::gen_seq<sizeof...(Ts)>());
}
// End of for_each_in_tuple implementation

template<typename ...Ts>
struct lock_guard {
public:
  explicit lock_guard(Ts&... ts) : mutexes(ts...) {
    lock(ts...);
  }

  lock_guard(Ts&... ts, std::adopt_lock_t) : mutexes(ts...) {}

  ~lock_guard() {
    for_each_in_tuple(mutexes, [](auto &m) { m.unlock(); } );
  }

  lock_guard(const lock_guard&) = delete;
  lock_guard& operator=(const lock_guard&) = delete;

private:
  tuple<Ts&...> mutexes;
};

template<typename T>
struct lock_guard<T> {
public:
  typedef T mutex_type;

  explicit lock_guard(T& _Mtx) : mtx(_Mtx) {
     mtx.lock();
  }
```

```
  lock_guard(T& _Mtx, std::adopt_lock_t) : mtx(_Mtx) {}

  ~lock_guard() {
     mtx.unlock();
  }

  lock_guard(const lock_guard&) = delete;
  lock_guard& operator=(const lock_guard&) = delete;

private:
  T& mtx;
};

template<>
struct lock_guard<> {
  explicit lock_guard()  {}
  lock_guard(std::adopt_lock_t) {}
  ~lock_guard() {}
  lock_guard(const lock_guard&) = delete;
  lock_guard& operator=(const lock_guard&) = delete;
};
```

Feature test macro

For the purpose of SG10, we recommend the feature test macro `__cpp_lib_variadic_lock_guard`